

ポートフォリオ

名前 廣本 勝志

学校 福岡情報ITクリエイター専門学校

趣味 自炊、映像作品鑑賞、筋トレ

GithubURL <https://github.com/ma-si-419>

スキル：C++ 2年

C# 1年

HLSL 半年

Git 2年

Unity 1年



最終制作作品

- ▶ 作品名 デカヘッド
- ▶ ジャンル 3D格闘ゲーム
- ▶ 開発環境 C++,DxLib
- ▶ 対応機種 Windows
- ▶ 制作期間 3か月
- ▶ 制作時期 2024年10月～
- ▶ 担当 モデル以外すべて



デカヘッド 企画

- ▶ ジャンル:3D格闘ゲーム
- ▶ 地上と空中を自在に動き
- ▶ エネルギー弾や近接攻撃などを駆使して戦う



←格闘

遠距離攻撃→



100

- 

床にぶつかった時の処理←



デカヘッド カメラワーク

- ▶ 画面に動きを出すために、キャラクターを画面中央に固定するのではなく、画面全体をキャラクターが動き回るようなカメラワークを実装。
- ▶ 常に敵の場所がわかるように敵とプレイヤーの間の座標をターゲット座標としている



デカヘッド カメラワーク

- ▶ キャラクターが高速移動するような際にカメラを一瞬だけおいていくことによってキャラクターの速度感をプレイヤーが感じやすいように工夫をした。
- ▶ 瞬間移動の際にはカメラをワープさせ、高速移動の際にはカメラを早く動かし追いつくことによってそれぞれの動きの違いを感じるようなカメラに調整した。



デカヘッド 駆け引き

- ▶ 攻撃を後ろから受けた際にはガードができないやダメージ量などを変化させることで単調なゲームにならないように調整。
- ▶ 相手の攻撃をステップで回避してから攻撃するなどただ攻撃するだけでなく、プレイヤーに相手との駆け引きの楽しさを味わってもらうように工夫をした。



デカヘッド 駆け引き

- ▶ 気力を消費しなければダッシュや必殺技を発動できないようにすることによって、攻撃をやめて気力をチャージするのか、攻撃し続けてダメージを与えるのかをプレイヤーに考えさせることによって、単調なゲームにならないように工夫をした。



デカヘッド フレーム

- それぞれの攻撃に発生フレーム、全体フレーム、次の攻撃に移行できるフレームを実装し、後隙の長い技や発生の速い技など各技に特徴を持たせられるような設計にした。

攻撃の情報をまとめたファイル

攻撃の名	ダメージ	発生フレーム	発生時の攻撃	プレイヤー攻撃の移動時間	移動	ヒット時の攻撃が格闘次にどの攻撃手の状態を使うアニメーション
Low1	3	40	12	24	3	0 弱 1 Low2 無 LowAttack1
Low2	3	40	12	24	1	0 弱 1 Low3 無 LowAttack2
Low3	3	40	12	24	1	0 弱 1 Low4 無 LowAttack3
Low4	3	40	12	24	1	0 弱 1 Low5 無 LowAttack4
Low5	5	50	12	34	1	0 中 1 Low6 無 LowAttack5
Low6	3	50	12	24	1	0 1 弱 1 Low7 中 LowAttack6
Low7	5	40	12	24	1	0 0 上吹っ飛び 1 Low8 無 LowAttack7
Low8	7	60	27	45	0.5	0 1 下吹っ飛び 1 None 吹っ飛び LowAttack8
Teleportat	5	60	30	40	0.5	0 1 吹っ飛び 1 None 吹っ飛び MiddleChargeAttack
Energy1	1	30	10	15	0	5 0 弱 0 Energy2 無 EnergyAttackRight
Energy2	1	30	10	15	0	5 0 弱 0 Energy3 無 EnergyAttackLeft
Energy3	1	30	10	15	0	5 0 弱 0 Energy4 無 EnergyAttackRight
Energy4	1	30	10	15	0	5 0 弱 0 Energy5 無 EnergyAttackLeft
Energy5	1	30	10	15	0	5 0 弱 0 Energy6 無 EnergyAttackRight
Energy6	1	30	10	15	0	5 0 弱 0 None 無 EnergyAttackLeft
Upper	5	40	10	30	1	0 0 上吹っ飛び 1 Low1 無 UpperAttack
Stan	5	40	10	30	1	0 0 中スタン 1 Low1 無 StanAttack
LegSweep	5	40	10	30	1	0 0 下スタン 1 Low1 無 LegSweepAttack
UpCharge	5	40	15	30	4	0
MiddleCh	5	40	15	30	4	0
DownCha	5	40	15	30	4	0
EnergyCh	3	40	25	30	0	5
DashAttac	5	40	15	30	2	0

キャラクター	体力	開始時の攻撃力	防御力	移動速度	チャージ追撃回数	必殺技1の エフェクトコスト	ダメージ値当たり判定種類	発生フレーム	硬直フレーム	必殺技2の エフェクトコスト	ダメージ値当たり判定種類	発生フレーム	硬直フレーム				
マウス	30000	30000	150	50	1.5	280	5 キグルミラLaser	30000	50	5 ビーム	20	30 キグルミラLaser	30000	60	0 格闘	10	20
ブルーヘッ	50000	10000	250	150	0.8	120	2 ブルービーLaser	30000	35	5 気弾	20	30 突撃 Laser	40000	60	0 投げ	10	20
ビッグボー	40000	20000	200	100	0.8	80	3 びっぐびーLaser	20000	30	5 ビーム	20	30 なげとばしLaser	30000	45	0 体当たり	10	20

キャラクターのステータス

デカヘッド その他

- ▶ Singleton, Stateパターンなどを使用することによってプログラムの可読性を上げることができた

プレイヤーのStateパターン

```
class CharacterStateBase : public StateBase
{
public:
    enum class CharacterStateKind
    {
        kIdle, //何もしていない状態
        kMove, //移動状態
        kJump, //ジャンプ状態
        kGuard, //ガード状態
        kNormalAttack, //通常攻撃状態
        kSpecialAttack, //必殺技状態
        kRush, //突撃状態
        kHitAttack, //攻撃を受けた時
        kButtonBashing, //敵との押し合い
        kCharge, //チャージ状態
        kStateKindNum //Stateの数
    };

public:
    CharacterStateBase(std::shared_ptr<Character> character);
    virtual void OnCollide(std::shared_ptr<Collidable> collider);
    CharacterStateKind GetKind() { return m_kind; }

    //次のState
    std::shared_ptr<CharacterStateBase> m_pNextState;

protected:
    virtual void ChangeState(std::shared_ptr<CharacterStateBase> nextState);
    std::shared_ptr<MyEngine::InputData> GetCharacterInput();

    /// <summary>
    /// 対戦相手の座標を取得する
    /// </summary>
}
```

当たり判定の管理を行う Singletonクラス

```
Physics::Physics()
{
}

void Physics::Entry(std::shared_ptr<Collidable> col)
{
    // 登録
    bool found = (std::find(m_collidables.begin(), m_collidables.end(), col) != m_collidables.end());
    if (!found)
    {
        m_collidables.emplace_back(col);
    }
    // 既に登録されていたらエラー
    else
    {
        assert(0 && "指定のcollidableは登録済みです。");
    }
}

void Physics::Exit(std::shared_ptr<Collidable> col)
{
    // 登録解除
    bool found = (std::find(m_collidables.begin(), m_collidables.end(), col) != m_collidables.end());
    if (found)
    {
        m_collidables.remove(col);
    }
    // 登録されてなかったらエラー
    else
    {
        assert(0 && "指定のcollidableが登録されていません。");
    }
}

void Physics::Update()
{
    // 仮処理
    for (auto& item : m_collidables)
    {
        item->m_nextPos = (item->m_rigidbody.GetPos() + item->m_rigidbody.GetVel());
    }
}
```

制作実績1 2024年6月~8月(3か月)

- ▶ 作品名 KOUMOTTO
 - ▶ ジャンル 3Dバトルアクション
 - ▶ 開発環境 C++,DxLib
 - ▶ 対応機種 Windows
 - ▶ 制作機種 3か月
 - ▶ 制作時期 2024年6月初旬~2024年8月下旬
 - ▶ 担当 モデル以外すべて
- ▶ 頑張ったこと、学んだこと
 - ・初めての3Dゲーム制作
 - ・エフェクトの実装
 - ・Stateパターン



制作実績1 2023年11月~2月(3か月)

- ▶ 作品名 姫とサル
- ▶ ジャンル 2Dディフェンスアクション
- ▶ 開発環境 C++,DxLib
- ▶ 対応機種 Windows
- ▶ 制作機種 3か月
- ▶ 制作時期 2023年11月下旬~2024年2月中旬
- ▶ 担当 一部の画像以外すべて
- ▶ 頑張ったこと、学んだこと
 - ・初めてのゲーム制作
 - ・セーブデータの作成
 - ・外部データ読み込み
 - ・見た目にこだわる

